

OE-EC604C — Object Oriented Programming

Semester VI • 3L:0T:0P • 3 Credits

Module 1 Exam Notes: Programming Paradigm & OOP Concepts

CGEC | B.Tech ECE (2023–27) | *Biraj Sarkar*

Contents

1	Evolution of Programming Paradigm	2
1.1	Paradigms Evolution Timeline	2
1.2	Structured vs. Object-Oriented Development	2
2	Core Object-Oriented Programming Concepts	2
2.1	Objects and Classes	3
2.2	Data Encapsulation and Data Abstraction	3
2.3	Inheritance	3
2.4	Polymorphism	3
2.5	Dynamic Binding and Message Passing	4

1. Evolution of Programming Paradigm

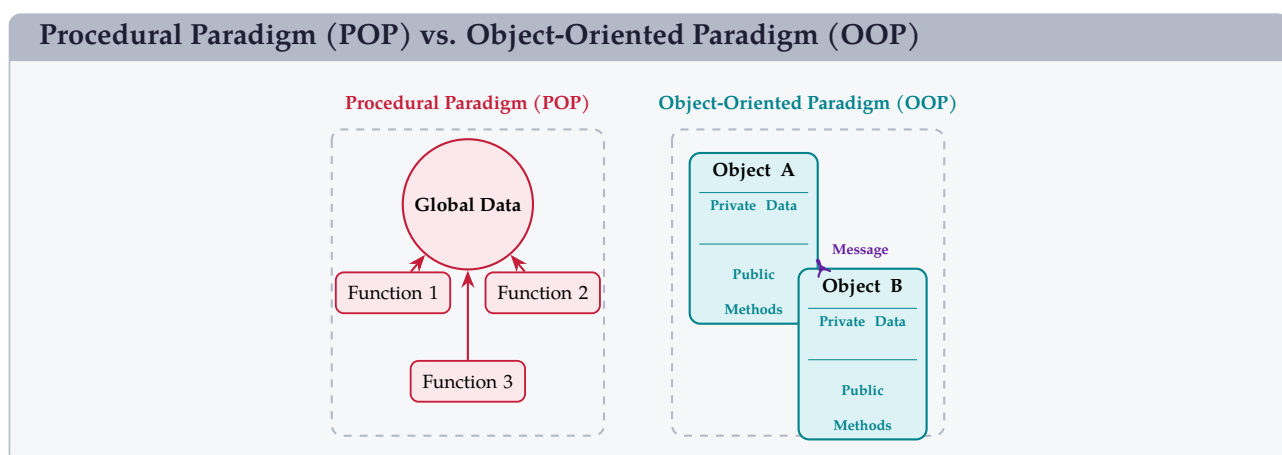
A programming paradigm defines the style, approach, and philosophy of designing and writing software code. As programs grew in size and complexity, paradigms evolved to improve code readability, reuse, and maintainability.

1.1 Paradigms Evolution Timeline

- 1. Unstructured Programming:** The earliest style where the program consists of a single continuous block of statements (often containing global variables) using jump statements like `goto` for flow control. It becomes difficult to maintain as code grows ("spaghetti code").
- 2. Procedural / Structured Programming:** Programs are organized into subroutines, modules, or functions. Flow control is structured using blocks like loops (`for`, `while`) and conditional statements (`if`, `switch`). Focus is on the **process or algorithm** rather than data.
- 3. Object-Oriented Programming (OOP):** A paradigm where programs are organized around **objects** (representing real-world entities) containing data (attributes) and methods (behaviors). Focus is on **data security and encapsulation** rather than procedures.

1.2 Structured vs. Object-Oriented Development

- **Structured/Procedure-Oriented Programming (POP):** Views a program as a sequence of procedural steps or function calls. Global data is vulnerable because it can be accessed and modified freely by any function.
- **Object-Oriented Programming (OOP):** Views a program as a collection of self-contained, interacting objects. Data is tied closely to the functions that operate on it, preventing accidental outside modification.



2. Core Object-Oriented Programming Concepts

OOP seeks to model software systems in terms of real-world entities. Below are the foundational concepts of OOP:

2.1 Objects and Classes

Class: A user-defined data type that acts as a blueprint, template, or prototype for creating objects. It defines the properties (data members) and actions (member functions) common to all objects of that class.

Object: A basic run-time entity of an object-oriented system that represents a real-world entity. It is an instance of a class, possessing its own state (data values) and behavior.

2.2 Data Encapsulation and Data Abstraction

Data Encapsulation: The mechanism of binding together code (member functions) and the data it manipulates (data members) into a single unit (class). It implements data hiding, keeping data safe from outside interference.

Data Abstraction: The act of representing essential features of a system without including the background details or implementation explanations. Users interact only with the interface, hiding complex execution.

2.3 Inheritance

Inheritance: The process by which objects of one class acquire the properties and behaviors of another class. It supports hierarchical classification and promotes **code reusability** by allowing new classes (derived classes) to extend existing ones (base classes).

Base Class (Parent Class): The class whose properties are inherited.

Derived Class (Child Class): The class that inherits from the base class.

2.4 Polymorphism

Polymorphism: The ability of an interface to take multiple forms (from Greek, meaning "many forms"). It allows a single function name, operator symbol, or interface to behave differently depending on the context.

There are two main classifications:

- **Compile-Time (Static) Polymorphism:** Resolved at compile time (e.g., function overloading, operator overloading).
- **Run-Time (Dynamic) Polymorphism:** Resolved at runtime during execution (e.g., virtual functions, method overriding).

2.5 Dynamic Binding and Message Passing

Dynamic Binding: The mechanism where the code associated with a given function call is not resolved until runtime. It is associated with inheritance and run-time polymorphism.

Message Passing: The communication process between objects. Objects interact by sending requests to execute methods of other objects, passing arguments and receiving results.

Quick Revision — Module I

Paradigm & OOP Concepts | OE-EC604C

Key Concepts at a Glance

Class Structure	<pre>class ClassName { private: // data public: // methods };</pre>
The 4 Pillars of OOP	Encapsulation (data hiding), Abstraction (interface separation), Inheritance (code reuse), Polymorphism (many forms).
Access Modifiers	private: Accessible only within class (default). public: Accessible from anywhere. protected: Accessible within class and child classes.

One-Line Definitions

- **Paradigm:** A structured method or style of programming defining program organization.
- **Class:** A user-defined logical blueprint defining data and member functions.
- **Object:** A physical instance of a class representing a real-world entity.
- **Encapsulation:** Wrapping data and code together into a single unit (class) to prevent global access.
- **Data Hiding:** Restricting direct access to internal data members using access specifiers.
- **Abstraction:** Hiding background implementation details and exposing only the interface.
- **Inheritance:** The mechanism of deriving new classes from existing classes to achieve code reuse.
- **Polymorphism:** The capability of a function, operator, or object to act differently in different contexts.
- **Dynamic Binding:** Resolving method calls at runtime rather than compile time.
- **Message Passing:** Communication model where objects invoke member functions of other objects.

Frequently Asked / Viva Questions

Q1. State the main limitation of Unstructured Programming.

Unstructured programming relies heavily on jump commands (like `goto`), making the control flow complex and highly difficult to trace, maintain, or debug.

Q2. What is the core difference in how POP and OOP view data?

Frequently Asked / Viva Questions (contd.)

Procedure-Oriented Programming (POP) views data as passive elements manipulated by active procedures. OOP views data as critical assets tied securely to active functions.

Q3. Define the difference between a Class and an Object.

A class is a logical template or blueprint (allocates no memory). An object is a physical instance of that template created in memory (allocates memory).

Q4. Why does OOP offer better data security than POP?

OOP hides data members inside classes using the `private` access specifier, preventing outside functions from modifying the variables. POP global data is accessible to all.

Q5. How does Encapsulation differ from Abstraction?

Encapsulation is the *wrapping* or *containment* of data (data hiding). Abstraction is the *hiding of complexity* by showing only the external interface.

Q6. What is the primary benefit of Inheritance?

Code reusability. It allows developer to extend base class properties without rewriting the code, reducing testing and development time.

Q7. Explain the term "Polymorphism" with a real-world example.

A single instruction performs different actions based on target. E.g., the "+" symbol performs addition for numbers, but performs concatenation for strings.

Q8. Define static binding vs. dynamic binding.

Static binding (early binding) resolves function calls at compile time. Dynamic binding (late binding) resolves function calls at runtime during execution.

Q9. What is the role of access specifiers in C++?

Access specifiers (`private`, `public`, `protected`) control the visibility and accessibility of class members from outside classes.

Q10. What is meant by the term "Message Passing"?

It is the process by which an object requests execution of a method belonging to another object, passing data parameters and receiving return values.

Q11. Can you have an object without a class in C++?

No. In C++, every object must belong to a predefined class which defines its structure and member specifications.

Q12. What is "Spaghetti Code" and in which paradigm does it commonly occur?

Spaghetti code is program code with highly convoluted control flow, usually caused by multiple jump statements (`goto`). It occurs in unstructured programming.

Comparison of POP and OOP Paradigms

Parameter	Procedure-Oriented (POP)	Object-Oriented (OOP)
Approach	Top-down design methodology.	Bottom-up design methodology.
Focus	Focuses on the algorithm/process.	Focuses on the data assets.
Data Access	Functions share global data (unsafe).	Objects hide data using private scope (secure).
Reusability	Limited to function calls.	Achieved via class inheritance.
Examples	C, Pascal, Fortran.	C++, Java, Python, C#.